

# Writing Lock-Free Code: A Corrected Queue

## Think in transactions, know who owns what, and use ordered atomic variables

By Herb Sutter

*Herb is a software development consultant, a software architect at Microsoft, and chair of the ISO C++ Standards committee. He can be contacted at [www.gotw.ca](http://www.gotw.ca).*

---

As we saw last month [1], lock-free coding is hard even for experts. There, I dissected a published lock-free queue implementation [2] and examined why the code was quite broken. This month, let's see how to do it right.

### Lock-Free Fundamentals

When writing lock-free code, always keep these essentials well in mind:

- Key concepts. Think in transactions. Know who owns what data.
- Key tool. The ordered atomic variable.

When writing a lock-free data structure, "to think in transactions" means to make sure that each operation on the data structure is atomic, all-or-nothing with respect to other concurrent operations on that same data. The typical coding pattern to use is to do work off to the side, then "publish" each change to the shared data with a single atomic write or compare-and-swap. [3] Be sure that concurrent writers don't interfere with each other or with concurrent readers, and pay special attention to any operations that delete or remove data that a concurrent operation might still be using.

Be highly aware of who owns what data at any given time; mistakes mean races where two threads think they can proceed with conflicting work. You know who owns a given piece of shared data right now by looking at the value of the ordered atomic variable that says who it is. To hand off ownership of some data to another thread, do it at the end of a transaction with a single atomic operation that means "now it's your's."

An ordered atomic variable is a "lock-free-safe" variable with the following properties that make it safe to read and write across threads without any explicit locking:

- Atomicity. Each individual read and write is guaranteed to be atomic with respect to all other reads and writes of that variable. The variables typically fit into the machine's native word size, and so are usually pointers (C++), object references (Java, .NET), or integers.
- Order. Each read and write is guaranteed to be executed in source code order. Compilers, CPUs, and caches will respect it and not try to optimize these operations the way they routinely distort reads and writes of ordinary variables.
- Compare-and-swap (CAS) [4]. There is a special operation you can call using a syntax like `variable.compare_exchange( expectedValue, newValue )` that does the following as an atomic operation: If `variable` currently has the value `expectedValue`, it sets the value to `newValue` and returns true; else returns false. A common use is `if(variable.compare_exchange(x,y))`, which you should get in the habit of reading as, "if I'm the one who gets to change variable from x to y."

Ordered atomic variables are spelled in different ways on popular platforms and environments. For example:

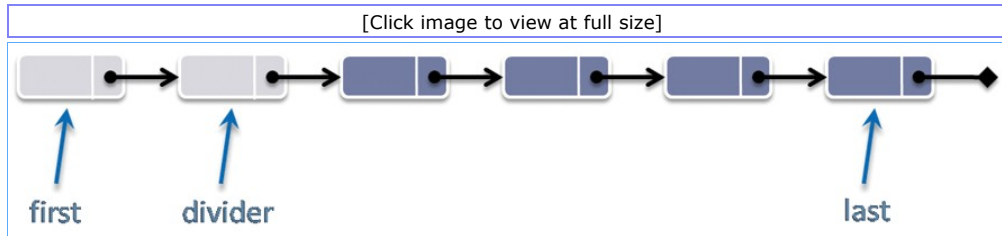
- `volatile` in C#/.NET, as in `volatile int`.
- `volatile` or `*Atomic*` in Java, as in `volatile int`, `AtomicInteger`.
- `atomic<T>` in C++0x, the forthcoming ISO C++ Standard, as in `atomic<int>`.

In the code that follows, I'm going to highlight the key reads and writes of such a variable; these variables should leap out of the screen at you, and you should get used to being very aware of every time you touch one.

If you don't yet have ordered atomic variables yet on your language and platform, you can emulate them by using ordinary but aligned variables whose reads and writes are guaranteed to be naturally atomic, and enforce ordering by using either platform-specific ordered API calls (such as Win32's *InterlockedCompareExchange* for compare-and-swap) or platform-specific explicit memory fences/barriers (for example, Linux *mb*).

## A Corrected One-Producer, One-Consumer Lock-Free Queue

Now let's tackle the lock-free queue using our essential tools. In this first take, to allow easier comparison with the original code in [2], I'll stay fairly close to the original design and implementation, including that I'll continue to make the same simplifying assumption that there is exactly one Consumer thread and one Producer thread, so that we can easily arrange for them to always work in different parts of the underlying linked list. In Figure 1, the first "unconsumed" item is the one after the *divider*. The consumer increments *divider* to say it has consumed an item. The producer increments *last* to say it has produced an item, and also lazily cleans up consumed items before the *divider*.



**Figure 1: The lock-free queue data structure.**

Here's the class definition, which carefully marks shared variables as being of an ordered atomic type (using C++ to most closely follow the original code in [2]):

```
template <typename T>
class LockFreeQueue {
private:
    struct Node {
        Node( T val ) : value(val), next(nullptr) { }
        T value;
        Node* next;
    };
    Node* first;
    atomic<Node*> divider, last;    // for producer only
                                   // shared
};
```

The constructor simply initializes the list with a dummy element. The destructor (in C# or Java, the *dispose* method) releases the list. In a future column, I'll discuss in detail why constructors and destructors of a shared object don't need to worry about concurrency and races with methods of the same object; the short answer for now is that creating or tearing down an object should always run in isolation, so no internal synchronization needed.

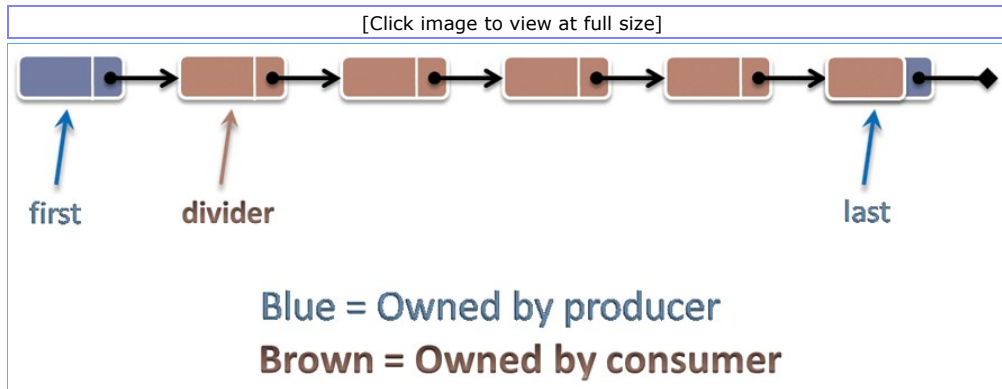
```
public:
    LockFreeQueue() {
        first = divider = last =
            new Node( T() );    // add dummy separator
    }
    ~LockFreeQueue() {
        while( first != nullptr ) {    // release the list
            delete first;
            first = first->next;
        }
    }
```

```

Node* tmp = first;
first = tmp->next;
delete tmp;
}
}

```

Next, we'll look at the key methods, *Produce* and *Consume*. Figure 2 shows another view of the list by who owns what data by color-coding: The producer owns all nodes before *divider*, the *next* pointer inside the *last* node, and the ability to update *first* and *last*. The consumer owns everything else, including the values in the nodes from *divider* onward, and the ability to update *divider*.



**Figure 2: Ownership rules of the road.**

## The Producer

*Produce* is called on the producer thread only:

```

void Produce( const T& t ) {
    last->next = new Node(t);    // add the new item
    last = last->next;           // publish it
    while( first != divider ) { // trim unused nodes
        Node* tmp = first;
        first = first->next;
        delete tmp;
    }
}

```

First, the producer creates a new *Node* containing the value and links it to the current *last* node. At this point, the node is not yet shared, but still private to the producer thread even though there's a link to it; the consumer will not follow that link unless the value of *last* says it may follow it. Finally, when all the real work is done—the node exists, its value is completely initialized, and it's correctly connected—then, and only then, do we write to *last* to "commit" the update and publish it atomically to the consumer thread. The consumer reads *last*, and either sees the old value (and ignores the new partly constructed element even if the *last->next* pointer might already have been set) or the new value that officially blesses the new node as an approved part of the queue, ready to be used.

Finally, the producer performs lazy cleanup of now-unused nodes. Because we always stop before *divider*, this can't conflict with anything the consumer might be doing later in the list. What if while we're in the loop, the consumer is consuming items and changing the value of *divider*? No problem: Each time we read *divider*, we see it either before or after any concurrent update by the consumer, both of which let the producer see the list in a consistent state.

## The Consumer

*Consume* is called on the consumer thread only:

```
bool Consume( T& result ) {
    if( divider != last ) {          // if queue is nonempty
        result = divider->next->value; // C: copy it back
        divider = divider->next;     // D: publish that we took it
        return true;                // and report success
    }
    return false;                   // else report empty
};
```

First, the consumer checks that the list is nonempty by atomically reading *divider*, atomically reading *last*, and comparing them. This one-time check is safe because although *last*'s value may be changed by the producer while we are running the rest of this method, if the check is true once, it will stay true even if *last* moves, because *last* never backs up; it can only move forward to publish new tail nodes—which doesn't affect the consumer, who only cares about the first node after the *divider*. If there is a valid node after *divider*, the consumer copies its value and then, finally, advances *divider* to publish that the queue item was removed.

Yes, we could eliminate the need to make the *last* variable shared: The consumer only uses the value of *last* to check whether there's another node after the *divider*, and we could instead have the consumer just test whether *divider->next* is non-null. That would be fine, and it would let us make *last* an ordinary variable; but if we do that, we must also remember that this change would make each *next* member a shared variable instead, and so to make it safe, we would also have to change *next*'s type to *atomic<Node\*>*. I'm leaving *last* as is for now to make it easier to compare this code with the original version in [2], which did use such a tail iterator to communicate between the two threads.

## Do Work, Then Publish

You might also have noticed that the original code in [2] did the equivalent of lines C (copy) and D (*divider* update) in the reverse order. You should always be alert and suspicious when you see code that tries to do things backwards: Remember, we're supposed to do all the work off to the side (line C) and only then publish that we did it (line D), as previously shown.

I'm sure someone is about to point out that we could actually get away with writing D then C in this code. Yes, but don't; it's a bad habit. It's true that, in this particular case and now that *divider* is an ordered atomic variable (which wasn't true in the original code), it just so happens that we could get away with writing D then C due to the happy accident of a detail of the implementation combining with a design restriction:

- We always maintain one placeholder *divider* element between the producer and the consumer, so "publishing" the change to *divider* what would otherwise be one step too soon, so that refers to an unconsumed node rather than to a consumed node, happens to be innocuous as long as we're only one step ahead.
- There's exactly one consumer thread, so multiple calls to *Consume* must run in sequence and can never get two steps ahead.

But it's still a bad habit to get into. It's not a good idea to cut corners by relying on "happy" accidents, especially because there's not much to be gained here from breaking the correct pattern. Besides, even if we wrote D then C now, it might be just another thing we'd have to change anyway next month, because...

## Coming Up

Next month, we will consider how to generalize the queue for multiple producer and consumer threads. Your homework: What new issues does this raise? What parts of the code we just considered would be broken in the presence of multiple consumers alone and why? What about multiple producers? What about both? Once you've discovered the problems, what would you need to change in the code and in the queue data structure itself to address them?

You have one month. Think of how you would approach it, and we'll take up the challenge when we return.

## Notes

[1] H. Sutter. "Lock-Free Code: A False Sense of Security" (*DDJ*, September 2008). ([www.ddj.com/cpp/209903274](http://www.ddj.com/cpp/209903274)).

[2] P. Marginean. "Lock-Free Queues" (*DDJ*, July 2008). ([www.ddj.com/208801974](http://www.ddj.com/208801974)).

[3] This is just like a canonical exception safety pattern—do all the work off to the side, then commit to accept the new state using nonthrowing operations only. "Think in transactions" applies everywhere, and should be ubiquitous in the way we write our code.

[4] Compare-and-swap (CAS) is the most widely available fundamental lock-free operation and so I'll focus on it here. However, some systems instead provide the equivalently powerful load-linked/store-conditional (LL/SC) instead.

## Acknowledgment

Thanks to Tim Harris for his comments on drafts of this article.